Universidad Católica de Santiago del Estero
Departamento Académico Rafaela
Cesana, Pedro – Lorenzo, Mateo - Rossa, Tomás

Informe del Trabajo Práctico: Motor de búsqueda de funciones similares vía AST

Carrera: Ingeniería en Informática.
Asignatura: Lenguajes Formales y Autómatas.
Profesores: Rossanigo, Ariel – Ferrero, Mariano.
Alumnos: Cesana, Pedro - Lorenzo, Mateo - Rossa, Tomás.
Entrega: 30 de Octubre del 2026.

Decisiones de diseño

- **Arquitectura de archivos:** separamos la lógica entre los archivos según mencionaba el enunciado, con el fin de facilitar la modificación de los módulos de código, a su vez, permitiendo trabajar por separado evitando conflictos por “pisarnos” en el código.
- **Umbral (0.70):** Es un valor inicial razonable porque permite detectar funciones con una similitud relativamente alta sin exigir que sean casi idénticas. Un umbral menor podría incluir coincidencias poco relevantes, mientras que uno mayor podría descartar funciones similares con pequeñas modificaciones. Es un valor orientativo que debe ajustarse según los resultados obtenidos.

Decision de metrica

- **Jaccard con n-gramas ($n = 3$):** Se elige n = 3 porque ofrece un equilibrio entre detectar coincidencias y conservar el contexto sintáctico.
  Por qué no menos (n = 1 o n = 2): los grupos son muy pequeños y pueden coincidir en funciones diferentes, porque muchos AST comparten tokens y estructuras comunes.
  Por qué no más (n = 4, n = 5, etc.): los grupos son más largos y específicos. Una pequeña modificación en la función puede cambiar varios grupos y reducir la similitud detectada.
  Por qué n = 3: permite comparar grupos cortos de tokens que conservan parte del contexto sintáctico sin exigir coincidencias demasiado largas.

Limitaciones conocidas

- **Cambios o agregados grandes de código (clones con muchas modificaciones):** Si a una función clonada se le agregan o borran varias líneas de código en el medio, la

secuencia de instrucciones cambia bastante y la herramienta deja de reconocerlas como similares.

- **Diferente forma de programar lo mismo (equivalencia de lógica):** Si dos funciones resuelven exactamente el mismo problema pero usando estructuras distintas (por ejemplo, usar un bucle *for* en una y recursión o una *list comprehension* en la otra), la herramienta genera estructuras internas completamente diferentes y no detecta que hacen lo mismo.
- **Reordenamiento grande de bloques de código:** Aunque la herramienta tolera pequeños cambios de orden entre líneas cercanas, si se cambia de lugar el orden de varios bloques completos (como mover condicionales *if/else* grandes de sitio), la secuencia cambia lo suficiente para que el puntaje de similitud baje.

Posibles mejoras

- **Normalización de Tipos:** Aprovechar las anotaciones de tipo (ast.arg.annotation) si existen en el código para enriquecer el token normalizado (ejemplo: VAR1:int vs VAR1:str).
- **Caché y Persistencia del AST Serializado:** Guardar en disco los tokens serializados de las funciones parseadas para evitar re-parsear y re-normalizar todo el repositorio en cada ejecución del CLI.